

tests/dynamic/suites/ chaos_suite.sh — operator guide

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:**
Authored for P10. Honest-SKIPs (topology_unsupported /
feature_disabled_by_config, §11.4.69) until the live dynamic stack +
injection hooks land; parse-clean + authored today.

Companion (§11.4.18) to the in-source documentation block
at the top of the script.

Overview

§11.4.85 / §11.4.169 **chaos / failure-injection** suite. It injects
real faults into the live dynamic stack and asserts the design's fail-
closed contract (design §10) holds:

- **C1** kill gluetun mid-request → branded **503 + Squid PID unchanged** (graceful_503 analyzer) AND **zero target packets on the real uplink** during the down window (no_leak analyzer) → recovery to 200 when the tunnel returns.
- **C2** drop the network to the tunnel → fail-closed 503, no leak.
- **C3** corrupt/delete the Redis vpn:status key → stale = DOWN = fail-closed 503 (never fall through to a leaking direct request).

Every verdict is an analyzer citing a captured artefact (§11.4.69).

Prerequisites

- Source library tests/dynamic/lib/analyzer_common.sh + committed tests/lib/evidence.sh; the graceful_503 + no_leak analyzers.
- curl, POSIX sh, date.
- For a real run: HELIX_DYNAMIC_STACK=1 + HELIX_PROXY_URL, plus operator/orchestrator-supplied **injection hook** commands (config injection §11.4.28; containers submodule §11.4.76 — never hardcoded podman/docker here):
 - HELIX_CHAOS_KILL_CMD — bring a profile's tunnel DOWN
 - HELIX_CHAOS_RESTART_CMD — bring it back UP

- `HELIX_CHAOS_REDIS_CORRUPT_CMD` — corrupt/delete
vpn:status:<p>
- `HELIX_CHAOS_CAPTURE_CMD` — start a real-uplink tcpdump
filtered to the target; prints the capture file path on stdout
- `HELIX_SQUID_PID_CMD` — print the current Squid PID (runtime
signature)

Usage examples

```
# Today (no live stack / no hooks): honest SKIP, exit 0:
bash tests/dynamic/suites/chaos_suite.sh
```

```
# P10 live run (hooks supplied via env):
HELIX_DYNAMIC_STACK=1 HELIX_PROXY_URL=http://127.0.0.1:34128 \
  HELIX_CHAOS_KILL_CMD=... HELIX_CHAOS_RESTART_CMD=... \
  HELIX_CHAOS_CAPTURE_CMD=... HELIX_SQUID_PID_CMD=... \
  bash tests/dynamic/suites/chaos_suite.sh
```

Env: `CHAOS_TARGET` (default `http://target-a.internal/`), `CHAOS_PROFILE` (default `profile-a`).

Edge cases

- **Live stack absent** → `topology_unsupported SKIP`, exit 0.
- **Injection hooks not configured** (`HELIX_CHAOS_KILL_CMD` / `HELIX_CHAOS_CAPTURE_CMD` empty) → `feature_disabled_by_config SKIP`, exit 0.
- **Cleanup is non-negotiable** — `_chaos_cleanup` runs the restart command in a trap ... EXIT INT TERM so the stack is left quiescent on every exit path (§11.4.14).
- **A 503 from a crashed proxy is not graceful** — the PID-unchanged check is the §11.4.108 runtime-signature that distinguishes graceful from crash.

§11.4.115 RED_MODE polarity

RED_MODE	What it asserts	PASS means
0 (default)	GREEN guard	fail-closed 503 + no leak + recovery 200
1	RED baseline	the pre-fix stack leaks / crashes / serves 200-on-down → defect reproduced

A `RED_MODE=1` run where the stack failed CLOSED (no defect) is a finding, not a pass.

Internal behaviour

- `#!/usr/bin/env bash`, POSIX-clean (`sh -n + bash -n`, §11.4.67).
- C1 path: snapshot PID, start capture, kill tunnel, probe (capture body + code), re-snapshot PID, build a `graceful_503` manifest, run the `graceful_503` + `no_leak` analyzers via `dyn_run_analyzer`, restart the tunnel, probe recovery.
- GREEN requires `graceful_503_rc == 0 AND no_leak_rc == 0 AND recovery == 200`. Evidence dir: `qa-results/p9-harness/chaos_<run-id>/`.

Related

- Analyzers `tests/dynamic/analyzers/graceful_503_analyzer.sh` + `no_leak_analyzer.sh` (the oracles this suite cites).
- `tests/dynamic/lib/analyzer_common.sh` — sourced base.
- `tests/dynamic/suites/run_all_suites.sh` — orchestrates this suite.
- Constitution §11.4.85 / §11.4.69 / §11.4.107 / §11.4.108 / §11.4.115 / §11.4.14 / §11.4.28 / §11.4.76; design §10/§13.

Last verified

2026-07-01 — run with no live stack: honest SKIP, exit 0; cleanup trap present; `sh -n + bash -n parse-clean`. Live fault-injection is exercised in **P10**.